You are a senior React Native / Expo engineer.

Your task is to transform the FitAI app into a production-level fitness app by fixing the Settings, Report, Discover, and Search systems to match real-world apps (like the reference images provided).

🚨 CURRENT PROBLEMS (MUST FIX)

1. Language change does NOT reflect across app
2. Report screen is static / fake
3. No real workout history tracking
4. No proper stats (calories, time, workouts)
5. Discover page has wrong UI (big search bar instead of icon-based navigation)
6. No search page with categorized results
7. No history icon navigation
8. No real images mapped to categories (abs, legs, full body, etc.)

🎯 GOAL

Build a **fully functional user system**:

- Global language system (real-time translation)
- Real workout history tracking
- Accurate report analytics
- Modern Discover UX (icon-based navigation)
- Dedicated Search page with visual categories
- Navigation to History from Discover

🔥 IMPLEMENTATION REQUIREMENTS

✅ 1. GLOBAL LANGUAGE SYSTEM (CRITICAL)

Create:

📁 context/LanguageContext.js

Features:

- Store selected language globally
- Persist using AsyncStorage
- Apply instantly across app

Create translation files:

📁 locales/en.json
📁 locales/fr.json
📁 locales/es.json
📁 locales/hi.json

Example:
```
{
"training": "Training",
"report": "Report",
"settings": "Settings",
"start": "Start",
"calories": "Calories"
}
```

Create hook:

useTranslation()

Usage:
```
t("training")
```

✅ 2. SETTINGS SCREEN (FULLY FUNCTIONAL)

When user changes language:
👉 Entire app must re-render in selected language

Also implement:

- Units toggle (metric/imperial)
- Difficulty toggle (affects workout generator later)
- Voice coach toggle
- Notifications toggle

Persist ALL settings using AsyncStorage

✅ 3. REAL WORKOUT HISTORY SYSTEM

Create:

📁 services/historyService.js

Functions:

```
saveWorkout({
date,
muscle,
level,
duration,
calories,
exercises
})
```

```
getWorkoutHistory()
```

```
getWeeklySummary()
```

```
getMonthlySummary()
```

✅ 4. REPORT SCREEN (LIKE REFERENCE APP)

Must include:

TOP STATS (HORIZONTAL)

- Total Workouts
- Total Calories
- Total Minutes

CALENDAR VIEW

- Highlight workout days
- Click date → show workouts

WEEKLY SUMMARY

Example:
May 25 – May 31

- 1 Workout
- 253 kcal

WORKOUT LIST

Each item:

- Image thumbnail
- Workout name (Chest Advanced)
- Time + Calories
- Date & time

✅ 5. DISCOVER SCREEN (UI FIX)

REMOVE:
❌ Big search bar

ADD:
✅ Top right icons:

- 🔍 Search icon
- 🕒 History icon

ON CLICK:

Search icon → navigate to Search Screen
History icon → navigate to Report/History Screen

✅ 6. SEARCH SCREEN (NEW PAGE)

📁 screens/SearchScreen.js

Must include:

TOP SEARCH INPUT

- Focused input field
- Cancel button

BODY FOCUS (WITH IMAGES)

Show horizontal categories:

- Full Body
- Abs
- Arms
- Legs
- Chest

Each must have:

- Real image
- Label

WORKOUT TYPE FILTERS

- Build Muscle
- Fat Burning
- Cardio
- Equipment

LEVEL FILTER

- Beginner
- Intermediate
- Advanced

DURATION FILTER

- <5 min
- 5–10 min

- 10–20 min

RESULT LOGIC

- Typing “legs” → show leg workouts
- Typing “abs” → show abs workouts

✅ 7. IMAGE MAPPING SYSTEM

Create:

📁 utils/getCategoryImage.js

Map:

"abs" → abs image
"chest" → chest image
"legs" → legs image
"fullbody" → full body image

Use high-quality URLs or local assets

✅ 8. NAVIGATION FLOW

Discover Screen:

- Search icon → SearchScreen
- History icon → ReportScreen

Search Screen:

- Select category → Workout list

Report Screen:

- Show all past workouts

✅ 9. DATA FLOW

After workout ends:

👉 Automatically call:
saveWorkout()

So report updates in real-time

✅ 10. UX DETAILS (IMPORTANT)

- Smooth transitions
- Clean card UI
- Real images (not placeholders)
- Responsive design
- No static dummy data

🚀 FINAL EXPECTED RESULT

After implementation:

✅ Language changes reflect instantly across app
✅ Report screen shows real workout history
✅ Calories, time, workouts tracked properly
✅ Discover screen matches modern UX (icons)
✅ Search screen feels like real fitness app
✅ Category images displayed correctly
✅ Navigation is smooth and logical

⚠️ IMPORTANT RULES

DO NOT:

- Hardcode data
- Use fake static UI
- Ignore persistence

DO:

- Use AsyncStorage
- Build modular components
- Keep system scalable

Now implement everything step-by-step.